

Creating a custom subagent in Claude Code is as simple as writing a Markdown file. The CLI uses YAML frontmatter to configure the agent's model, tool access, and routing behavior.

1. Choose a Location

Claude Code looks for subagent definitions in two specific directories:

- **Project-level:** <project-root>/claude/agents/ (ideal for codebase-specific agents you want to commit to version control).
- **User-level:** ~/.claude/agents/ (personal agents available globally across all your projects).

2. Define the Agent

Create a new .md file in your chosen directory (e.g., code-reviewer.md). The top of the file must contain a YAML configuration block, and the body below it becomes the system prompt. Here is an example of an agent hardcoded to use a faster model for a restricted task:

```
Markdown
---
name: code-reviewer
description: Scans code for security vulnerabilities like SQL injection and XSS. Use this agent when analyzing files for security risks.
tools: Read, Grep, Glob
model: haiku
---
You are a security auditor. Focus on identifying vulnerabilities and hardcoded secrets. Explain each issue and show the current code, but do not edit the files directly.
```

3. Configuration Breakdown

- **model:** This field assigns the specific model. You can use tier aliases (haiku, sonnet, opus, fable) or a full model ID (like claude-opus-5). If you omit this field entirely, the subagent defaults to inheriting whichever model is currently active in your main session.
- **name:** The internal identifier, which should be lowercase and use hyphens.
- **description:** This is the trigger. Claude reads this to decide when to automatically delegate a task to this subagent. Be explicitly clear about what scenarios should trigger it.
- **tools:** A comma-separated list of allowed tools. Restricting tools (omitting Write or Bash, for example) is the best way to ensure a subagent doesn't do anything destructive or out of scope.

4. Invoking the Subagent

Once the file is saved, the agent is immediately active. You can trigger it in two ways:

- **Automatic Delegation:** Ask your main session a question that strongly matches the description field (e.g., "Check my recent changes for security issues"). The main orchestrator will detect the match and spawn the subagent in the background.
- **Explicit Invocation:** Bypass the orchestrator's routing logic and call the agent directly by using `/agent:<name>` in your prompt (e.g., `/agent:code-reviewer review src/auth/`).